

#5. LLMs with Tabular Data

Instructor: **Hankook Lee** @ Efficient Learning Lab., Sungkyunkwan University

Lecture Overview

- This lecture covers **the use LLMs with tabular data**
 - **What LLMs bring to Tabular ML?**
 - **Tables as Language: Serialization and Prediction**
 - LIFT (2022), TableLLM (2023), TABLET (2023)
 - **LLMs as Semantic Components in Tabular ML**
 - CAAFE (2023), FeatLLM (2024), OCTree(2024), DeLTa (2025)
 - **LLMs for Tabular Anomaly Detection**
 - AnoLLM (2025), ReTabAD (2026), AutoAnoEval (2026)
 - **LLM-based tabular foundation models**
 - TABULA (2024)

Large Language Models (LLMs)

- LLMs are trained to understand and generate text at scale
 - Learn semantic relationships among words, concepts, and descriptions
 - Follow natural-language instructions for new tasks
 - Adapt to new problems from zero or a few demonstrations
 - Provide useful prior knowledge about real-world entities and concepts
- However, tables are not naturally written as text
 - A tabular row must be serialized in a language-compatible form
 - Numerical values and structured relationships may remain difficult for LLMs

Question:

Can the semantic capabilities of LLMs be useful for tabular data?

What LLMs bring to Tabular ML?

- In previous lectures, most tabular models mainly **learn from observed values and statistical patterns**
- LLMs can additionally **leverage textual semantics and prior knowledge**
 - **Semantic understanding of columns and values**
 - E.g., "BMI" and "Diagnosis Code" carry interpretable meanings
 - **Common and domain-related knowledge**
 - Cities, occupations, diseases, products, and institutions may provide useful context
 - **Natural-language task specification**
 - The label meanings and decision context can be stated directly in a prompt
 - **Zero-shot and few-shot adaptation**
 - Predictions may be made with little or no task-specific training data
 - **Semantic reasoning for anomaly detection**
 - Abnormality may depend on contextual meaning, not only unusual numerical values

What LLMs bring to Tabular ML?

- In previous lectures, most tabular models mainly learn from observed values and statistical patterns

- LLMs can

- Semantic

- E.g.,

- Common

- Cities

- Natural language task specification

- The label meanings and decision context can be stated directly in a prompt

- Zero-shot and few-shot adaptation

- Predictions may be made with little or no task-specific training data

- Semantic reasoning for anomaly detection

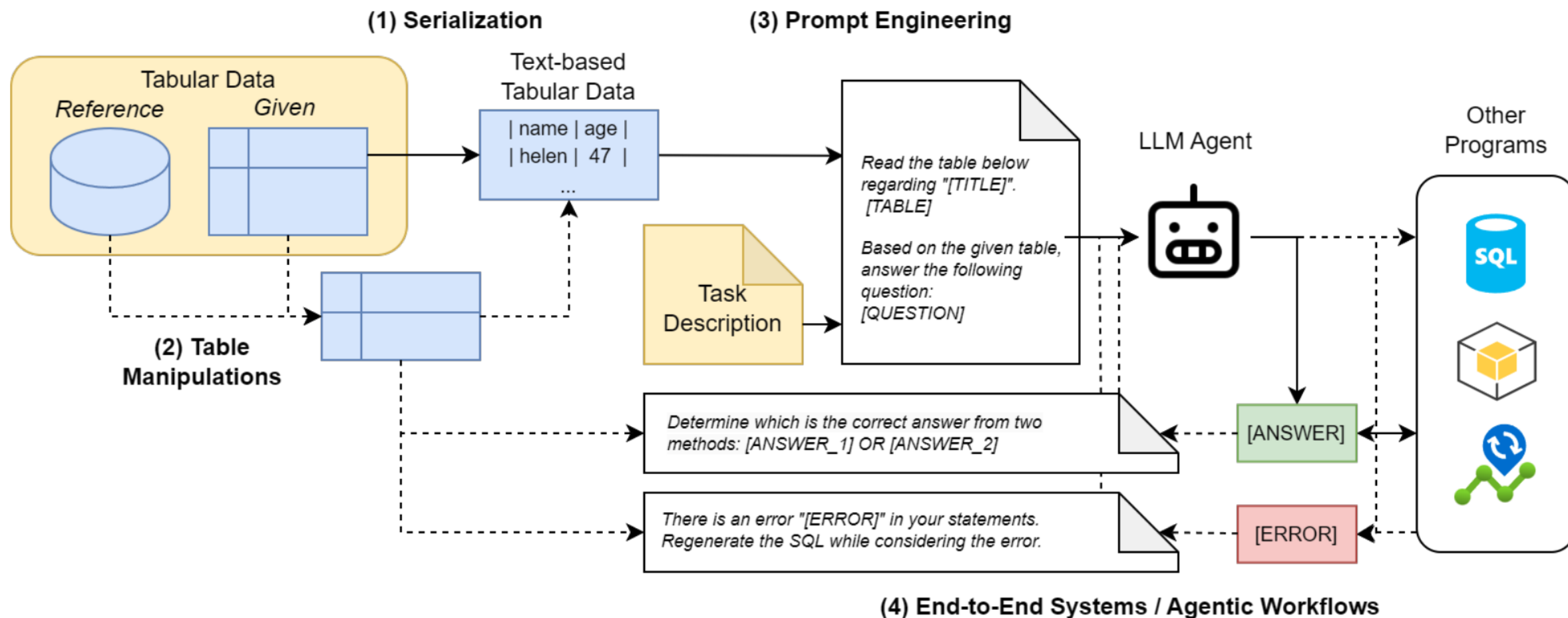
- Abnormality may depend on contextual meaning, not only unusual numerical values

**LLMs extend tabular ML
from learning numerical patterns
to reasoning over semantic context**

knowledge

context

LLM Workflow



Main Questions for This Lecture

- **Can an LLM directly predict from tabular data?**
 - Serialize rows as text and use the LLM as a predictor
- **Can an LLM improve another tabular predictor?**
 - Use semantic knowledge for feature engineering or hybrid pipelines
- **Can an LLM help detect semantically abnormal samples?**
 - Use likelihood scoring, restored context, or semantic evaluation
- **Can we build an LLM-based tabular foundation model?**
 - Pre-train on many tables and transfer to unseen tasks

Main Questions for This Lecture

- **Can an LLM directly predict from tabular data?**
 - Serialize rows as text and use the LLM as a predictor
- Can an LLM improve another tabular predictor?
 - Use semantic knowledge for feature engineering or hybrid pipelines
- Can an LLM help detect semantically abnormal samples?
 - Use likelihood scoring, restored context, or semantic evaluation
- Can we build an LLM-based tabular foundation model?
 - Pre-train on many tables and transfer to unseen tasks

Tables Must Become Text

- LLMs do not directly process tables as structured inputs
- Before prediction, **a table must be turned into a token sequence**
 - This step is called **serialization** (or linearization)
- The **serialization format is part of the model design**
 - It affects what structure the LLM can see and how many tokens are spent

Common Serialization Methods

Method	Description	Example	Papers that investigated this
DFLoader	Python code where a dictionary is loaded as a Pandas dataframe	<code>pd.DataFrame({ name:['helen'], age:[47] })</code>	Singha et al. (2023)
JSON	Row number as indexes, with each row represented as a dictionary of keys (column names) and values	<code>{"0": { "name": "helen", "age": "47"}}</code>	Singha et al. (2023); Sui et al. (2023b)
Data Matrix	Dataframe as a list of lists, where the first item is the column header	<code>[['', 'name', 'age'], [0, 'helen', 47]]</code>	Singha et al. (2023)
Markdown	Rows are line-separated, columns are separated by “ ”	<code> name age -- ----- ----: 0 helen 47 </code>	Singha et al. (2023); Liu et al. (2023e); Zhang et al. (2023d); Ye et al. (2023b); Zhao et al. (2023d); Sui et al. (2023b)
LaTeX	Rows are separated by “\\hline”, columns are separated by “&”	<code>\\hline helen & 47</code>	Jaitly et al. (2023)
X-Separated	Rows are line-separated, columns are separated by “,”, “\t”, “:”, etc.	<code>, name, age 0, helen, 47</code>	Singha et al. (2023); Narayan et al. (2022)
Attribute-Value Pairs	Concatenation of paired columns and cells {c : v}	<code>name:helen ; age:47</code>	Wang et al. (2023c)
HTML	HTML element for tabular data	<code><table><thead><tr><th></th> <th>name</th><th>age</th></tr> </thead><tbody><tr><th>0</th> <td>helen</td><td>47</td></tr> </tbody></table></code>	Singha et al. (2023); Sui et al. (2023c;b)
Sentences	Rows are converted into sentences using templates	<code>name is helen, age is 47</code>	Yu et al. (2023); Hegselmann et al. (2023); Gong et al. (2020); Dinh et al. (2022); Jaitly et al. (2023)

LIFT: Language Interface for Tabular Tasks

- **Key Idea:** Convert non-language ML tasks into **language-input** and **language-output** tasks
 - No architecture or loss modification
 - Show that LLMs can be fine-tuned through a natural-language interface

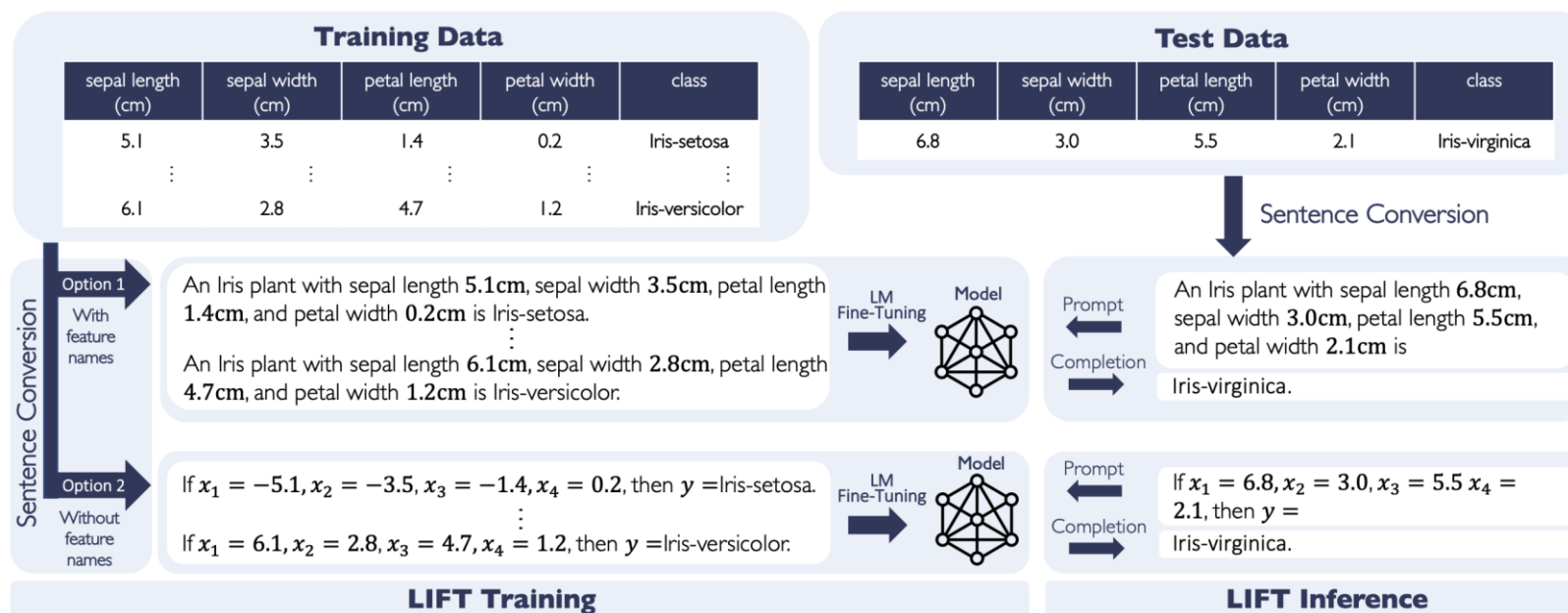


Figure 1: A high-level illustration of the Language-Interfaced Fine-Tuning (LIFT) framework.

TabLLM: Tabular Classification with LLMs

- **Key Idea:** Serialize a tabular row into a natural-language string with a short task description
 - Use LLMs for zero-shot or few-shot classification

1. Tabular data with k labeled rows

age	education	gain	income
39	Bachelor	2174	≤50K
36	HS-grad	0	>50K
64	12th	0	≤50K
29	Doctorate	1086	>50K
42	Master	594	

2. Serialize feature names and values into natural-language string with different methods

Manual Template

The age is 42. The education is Master. The gain is 594.

Table-To-Text

The person is 42 years old. She has a Master. The gain is 594 dollars.

LLM

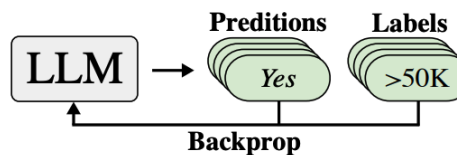
The person is 42 years old and has a Master's degree. She gained \$594.

3. Add task-specific prompt

Does this person earn more than 50000 dollars? Yes or no? Answer:

The age is 29. The education is Doctorate. The gain is 1086.
Does this person earn more than 50000 dollars? Yes or no?
Answer:

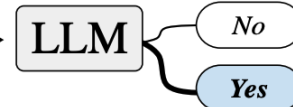
4a. Fine-tune LLM using labeled examples



4b. Use LLM for prediction on unlabeled examples

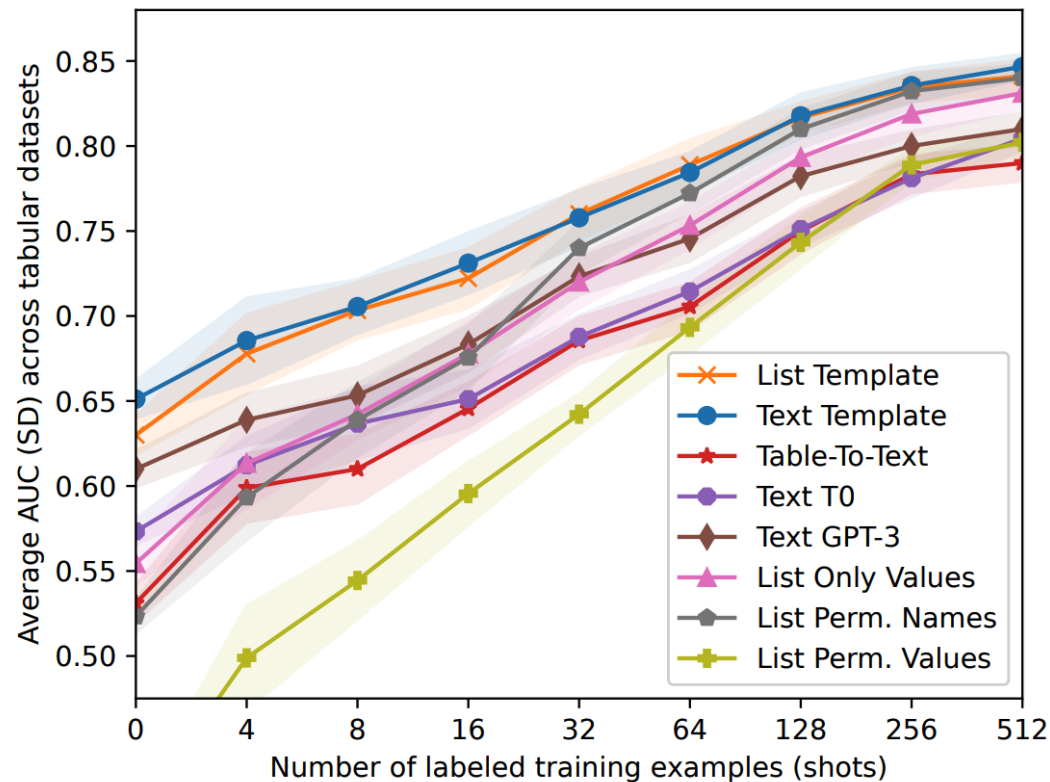
The age is 42. The education is Master. The gain is 594.

Does this person earn more than 50000 dollars? Yes or no?
Answer:



TabLLM: Tabular Classification with LLMs

- **Key Idea:** Serialize a tabular row into a natural-language string with a short task description
 - Show that **input serialization matters**



- **List Template:** A list of column names and feature values. We fixed an arbitrary ordering of the columns.
- **Text Template:** An textual enumeration of all features as “The *column name* is *value*.” (see Figure 1).
- **Table-To-Text:** We use an LLM fine-tuned on a table-to-text generation task from HuggingFace (Narrativaai/bloom-560m-finetuned-totto-table-to-text). To ensure that the serialization includes all data we hand each column-value tuple to the model separately and concatenate the outputs.
- **Text T0:** We use the LLM T0 with 11B parameters (bigscience/T0pp) (Sanh et al., 2022). We split up a row into pairs of two column-value tuples. We send them to LLM separately with the prompt “Write this information as a sentence:” and combine the outputs.
- **Text GPT-3:** We use GPT-3 (engine *text-davinci-002*) accessible through an API (Ouyang et al., 2022). GPT-3 was able to serialize all features at once, so we use a list of all features with the prompt “Rewrite all list items in the input as a natural text.” as input. We guide the output with “The {person, car, patient} is”.

TABLET: Learning from Instructions

- **Key Idea:** Evaluate how **natural-language instructions** help LLMs solve tabular prediction tasks
 - Show that **instructions can improve zero/few-shot tabular prediction**

Original Prompt

Determine if Whooping cough should be included in a differential diagnosis for the patient. Answer with one of the following: no | yes. Here are the patient's responses to questions about their symptoms.

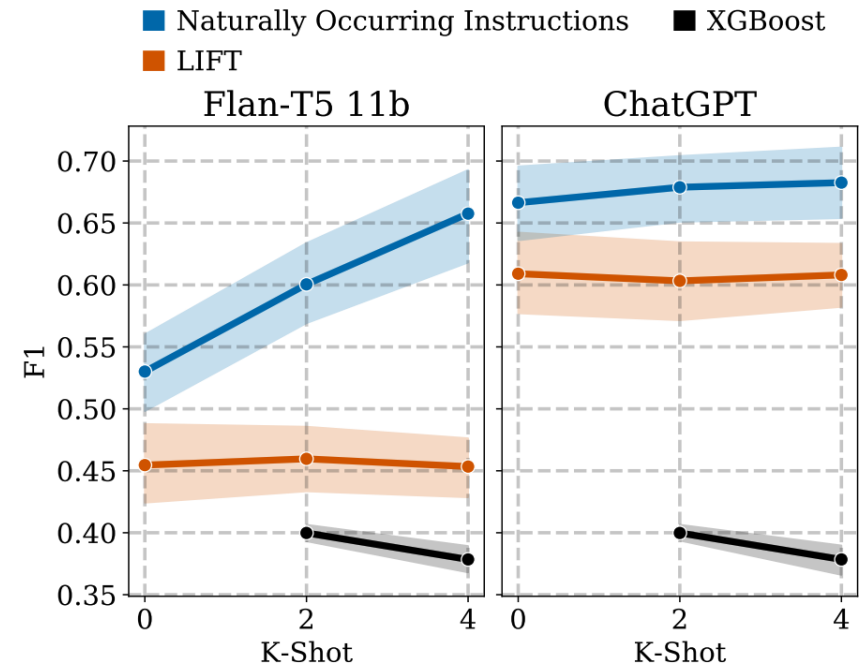
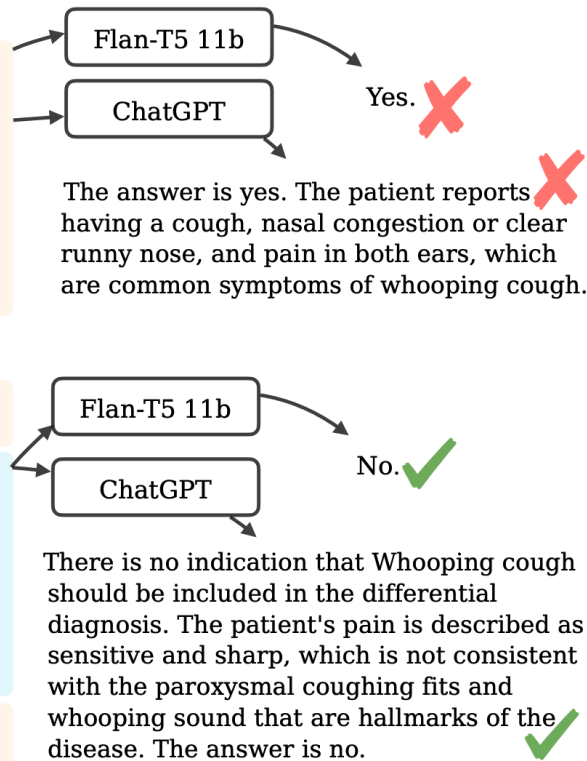
- Characterize your pain: sensitive and sharp
- Do you feel pain somewhere?: left ear and right ear
- Do you have a cough?: yes
- Do you have nasal congestion or a clear runny nose?: yes
- Where is the affected region located?: left iliac fossa

+ Naturally Occuring Instructions

....Answer with one of the following: no | yes.

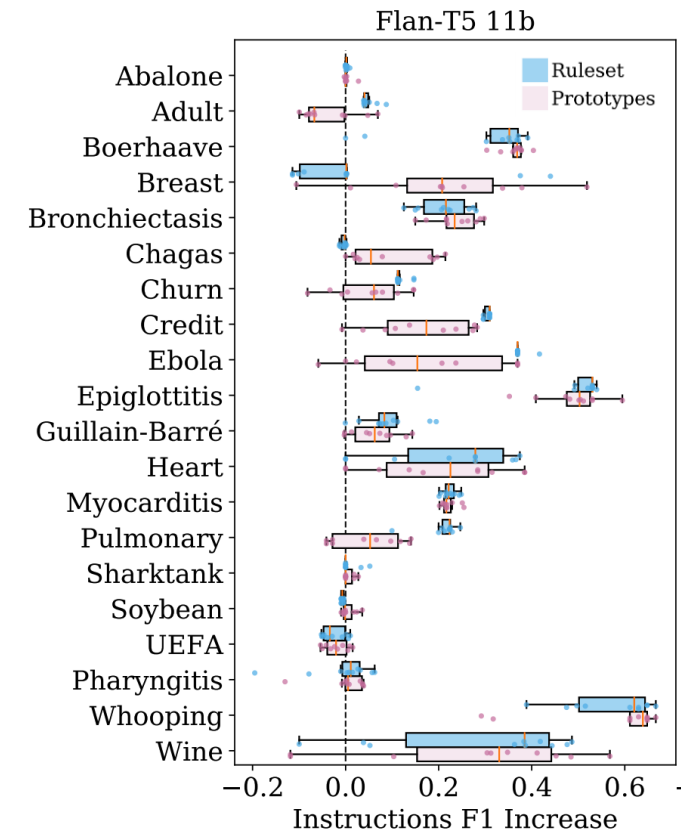
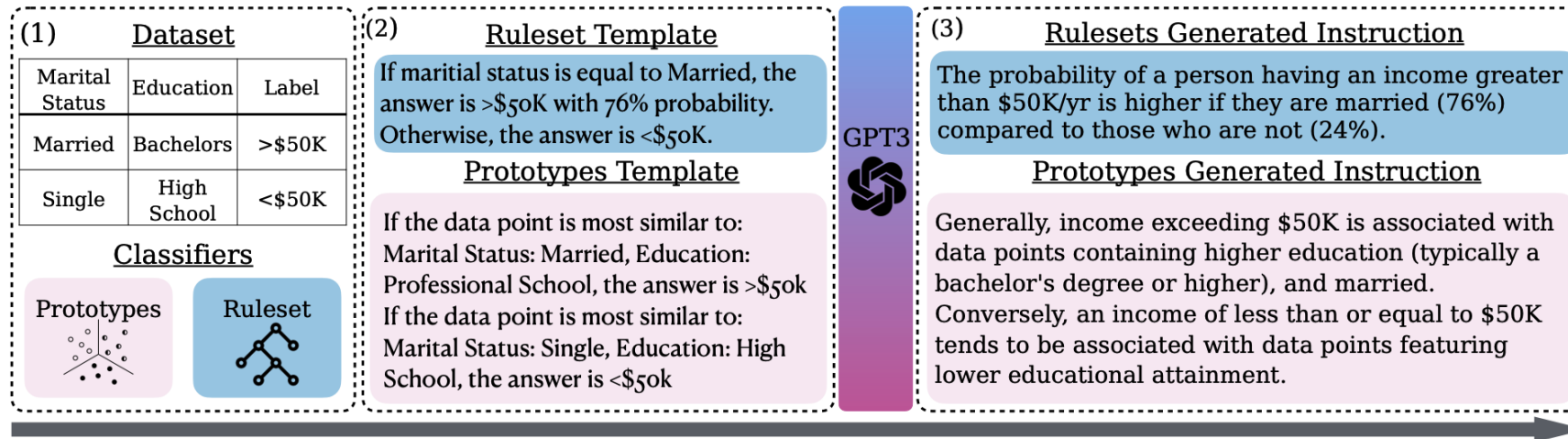
B. pertussis (whooping cough) invades respiratory mucosa, increasing the secretion of mucus... After 10 to 14 days, the paroxysmal stage begins with an increase in the severity and frequency of the cough. Repeated bouts of ≥ 5 rapidly consecutive forceful coughs occur during a single expiration and are followed by the whoop... Paroxysmal coughing may recur for months. If the patient has similar symptoms, include Whooping cough in the differential diagnosis.

Here are the patient's response to questions about their symptoms....



TABLET: Learning from Instructions

- **Key Idea:** Evaluate how **natural-language instructions** help **LLMs** solve tabular prediction tasks
 - Show that **instructions can improve zero-shot tabular prediction**
 - **Generated instructions** can be also useful



Summary: Serialization & Prediction

- **Main strategy:** Convert tabular prediction into language prediction
 - Strength: uses semantic prior knowledge and task descriptions
 - Weakness: inference cost, context limits, serialization sensitivity
- When using LLMs, **input design matters**
 - Row serialization
 - Task description
 - Natural-language instructions
- **LLMs can directly solve tabular prediction problems,** but this is **not always practical**
 - LLMs may struggle with numerical values, long tables, and inference cost

Main Questions for This Lecture

- Can an LLM directly predict from tabular data?
 - Serialize rows as text and use the LLM as a predictor
- **Can an LLM improve another tabular predictor?**
 - Use semantic knowledge for feature engineering or hybrid pipelines
- Can an LLM help detect semantically abnormal samples?
 - Use likelihood scoring, restored context, or semantic evaluation
- Can we build an LLM-based tabular foundation model?
 - Pre-train on many tables and transfer to unseen tasks

Beyond Direct LLM Prediction

- **Direct LLM prediction is often impractical**
 - Serialize every row & query LLM for each prediction
 - Sensitive to prompt length and input format
 - Expensive for large-scale inference
- **Alternative: Use LLMs as semantic components**
 - Generate useful features
 - Improve feature-generation rules
 - Refine decision-tree rules

Let standard ML models perform final prediction
instead of asking the LLM to predict every row

Semantic Feature Engineering with LLMs

- **Why feature engineering?**

- Tabular performance often depends on meaningful feature transformations
- LLMs can use column names, value meanings, and task descriptions
- This allows feature generation beyond fixed search spaces

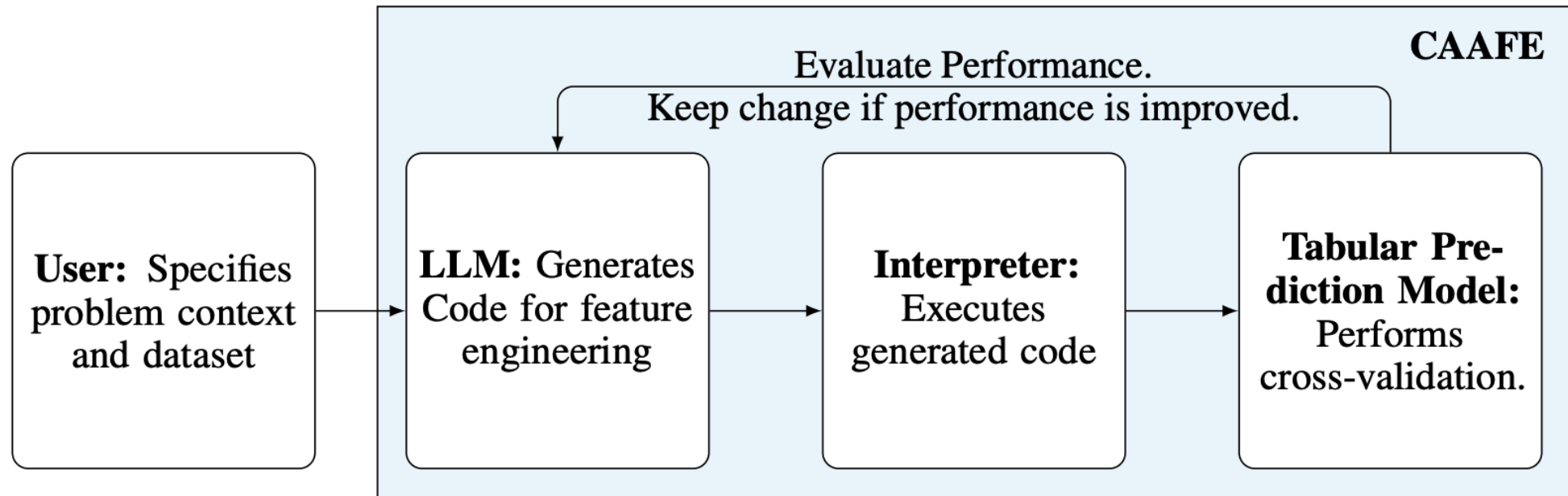
- **Common patterns**

- LLM proposes feature transformations
- A standard ML model evaluates them
- Useful features are kept for final prediction

LLMs can act as feature engineers that translate semantic knowledge into usable columns

CAAFE: Feature Engineering via LLM

- **Key Idea:** Use an LLM to generate semantically meaningful features
 - Provide dataset context, column names, and task description
 - LLM generates Python code for new features



CAAFE: Feature Engineering via LLM

- **Key Idea:** Use an LLM to generate semantically meaningful features
 - Provide dataset context, column names, and task description
 - LLM generates Python code for new features

```
Dataset description: Tic-Tac-Toe Endgame database
This database encodes the complete set of possible board
configurations at the end of tic-tac-toe games, where "x" is
assumed to have played first. The target concept is "win for x" (i
.e., true when "x" has one of 8 possible ways to create a "three-
in-a-row").
```

```
# ('number-of-x-wins', 'Number of ways x can win on the board')
# Usefulness: Knowing the number of ways x can win on the board can be useful in
# predicting whether x has won the game or not.
# Input samples: 'top-left-square': [2, 2, 1], 'top-middle-square': [1, 2, 0], ...
df['number-of-x-wins'] = ((df['top-left-square']==1) & (df['top-middle-square']==1) & (df
['top-right-square']==1)).astype(int) + ((df['middle-left-square']==1) & (df['middle
-middle-square']==1) & (df['middle-right-square']==1)).astype(int) [...]
```

```
Iteration 1
Performance before adding features ROC 0.888, ACC 0.700.
Performance after adding features ROC 0.987, ACC 0.980.
Improvement ROC 0.099, ACC 0.280. Code was executed and changes to df
retained.
```

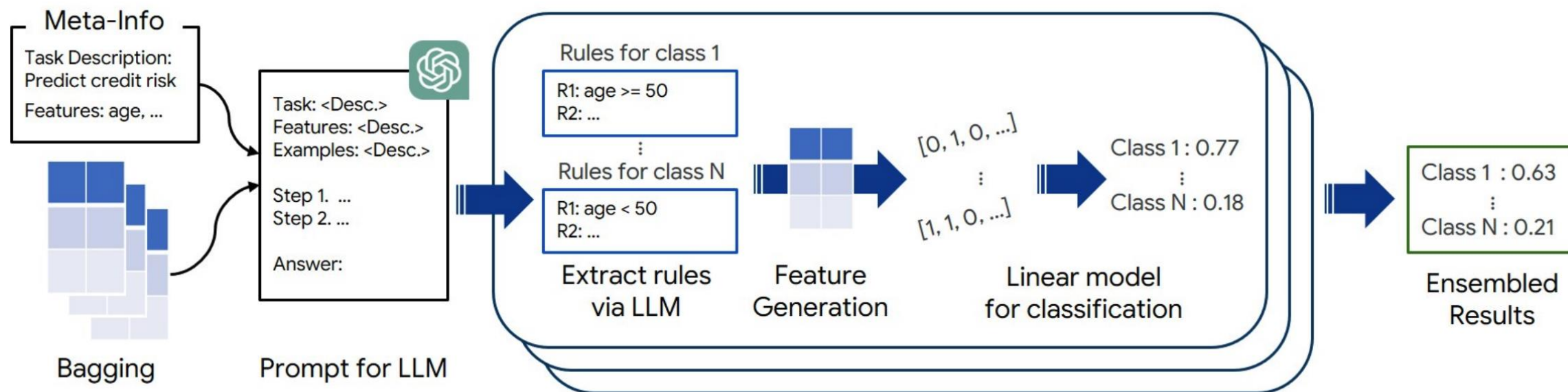
CAAFE: Feature Engineering via LLM

- **Key Idea:** Use an LLM to generate semantically meaningful features
 - Generated features can improve performance in downstream tasks
 - Better LLMs can generate better features

	TabPFN		
	No Feat. Eng.	CAAFE (GPT-3.5)	CAAFE (GPT-4)
airlines	0.6211 ±.04	0.619 ±.04	0.6203 ±.04
balance-scale [R]	0.8444 ±.29	0.844 ±.31	0.882 ±.26
breast-w [R]	0.9783 ±.02	0.9809 ±.02	0.9809 ±.02
cmc	0.7375 ±.02	0.7383 ±.02	0.7393 ±.02
credit-g	0.7824 ±.03	0.7824 ±.03	0.7832 ±.03
diabetes	0.8427 ±.03	0.8434 ±.03	0.8425 ±.03
eucalyptus	0.9319 ±.01	0.9317 ±.01	0.9319 ±.00
jungle_chess..	0.9334 ±.01	0.9361 ±.01	0.9453 ±.01
pc1	0.9035 ±.01	0.9087 ±.02	0.9093 ±.01
tic-tac-toe [R]	0.6989 ±.08	0.6989 ±.08	0.9536 ±.06
⟨Kaggle⟩ health-insurance	0.5745 ±.02	0.5745 ±.02	0.5748 ±.02
⟨Kaggle⟩ pharyngitis	0.6976 ±.03	0.6976 ±.03	0.7078 ±.04
⟨Kaggle⟩ kidney-stone	0.7883 ±.04	0.7873 ±.04	0.7903 ±.04
⟨Kaggle⟩ spaceship-titanic	0.838 ±.02	0.8383 ±.02	0.8405 ±.02

LLM as Few-Shot Feature Engineer

- **Key Idea:** Use LLMs to generate predictive features from few-shot examples
 - LLM proposes rule-based features → train a simple predictor
 - Different LLM-generated rules capture different semantic hypotheses → Ensemble multiple predictors



LLM as Few-Shot Feature Engineer

- **Key Idea:** Use LLMs to generate predictive features from few-shot examples

Task: <Task description>

Features: <Feature descriptions>

Examples: <Serialized training examples>

Let's first understand the problem and solve the problem step by step.

Step 1. Analyze the causal relationship or tendency between each feature and task description based on general knowledge and common sense within a short sentence.

Step 2. Based on the above examples and Step 1's results, infer 10 different conditions per answer, following the format below. The condition should make sense, well match examples, and must match the format for [condition] according to value type.

Format for Response:

10 different conditions for class [Class name]:

- [Condition]

...

Format for [Condition]:

For the categorical variable only,

- [Feature] is in [List of categories]

For the numerical variable only,

- [Feature] (> or >= or < or <=) [Value]

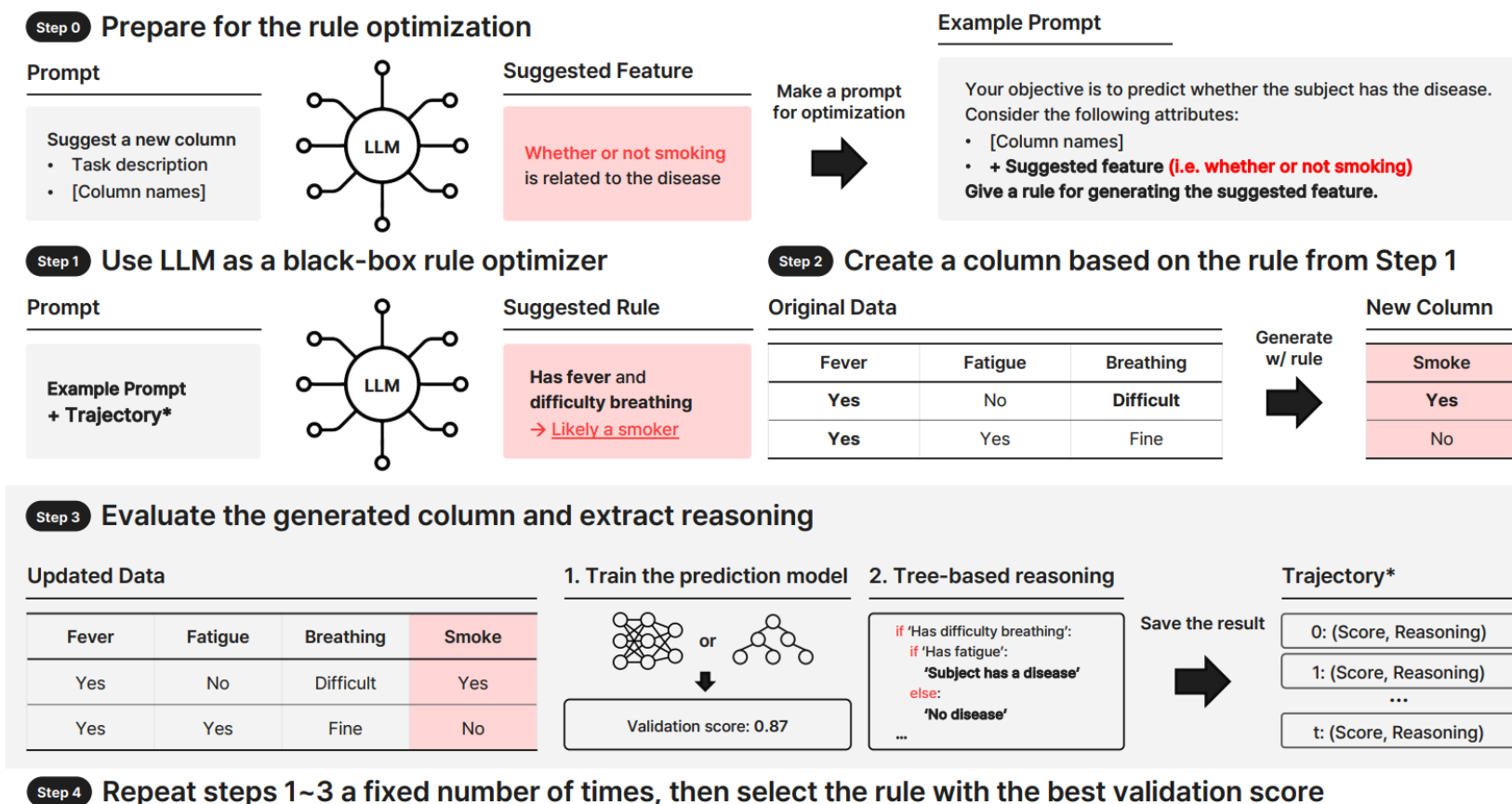
- [Feature] is within range of [Value_start, Value_end]

Answer:

Step 1.

Feature Generation with Decision Tree

- **Key Idea:** Use LLMs as optimizers for feature-generation rules with decision tree reasoning



Feature Generation with Decision Tree

- **Key Idea:** Use LLMs as optimizers for feature-generation rules with decision tree reasoning

with language descriptions

Method	LLM	Tesla [†]	Enefit [†]	Disease*	Clinical*	Academic*
<i>XGBoost</i> [11]						
Baseline	-	6.61	8.00	28.09 $\pm$ 7.9	46.27 $\pm$ 5.0	14.15 $\pm$ 0.6
OCTree	Llama 2	5.56 (15.9%)	8.00 (0.0%)	26.19 $\pm$ 7.2 (6.8%)	45.07 $\pm$ 4.1 (2.6%)	14.11 $\pm$ 0.5 (0.3%)
OCTree	GPT-4o	5.48 (17.1%)	7.82 (2.3%)	25.72$\pm$6.6 (8.4%)	43.75$\pm$4.4 (5.4%)	13.74$\pm$0.1 (2.9%)
<i>MLP</i> [31]						
Baseline	-	7.41	33.53	38.10 $\pm$ 3.6	41.77 $\pm$ 1.7	14.41 $\pm$ 0.8
OCTree	Llama 2	5.23 (29.4%)	29.99 (10.6%)	32.86 $\pm$ 5.7 (13.7%)	39.80 $\pm$ 2.3 (4.7%)	14.26 $\pm$ 0.7 (1.0%)
OCTree	GPT-4o	5.01 (32.4%)	21.68 (35.3%)	30.95$\pm$5.8 (18.8%)	39.25$\pm$0.5 (6.0%)	14.22$\pm$0.5 (1.3%)
<i>HyperFast</i> [32]						
Baseline	-	N/A	N/A	28.57 $\pm$ 10.0	43.64 $\pm$ 1.1	14.67 $\pm$ 0.7
OCTree	Llama 2	N/A	N/A	28.10 $\pm$ 9.2 (1.6%)	41.45$\pm$1.7 (5.0%)	14.49$\pm$0.5 (1.2%)
OCTree	GPT-4o	N/A	N/A	27.14$\pm$3.8 (5.0%)	42.00 $\pm$ 1.5 (3.8%)	14.49$\pm$0.5 (1.2%)

Feature Generation with Decision Tree

- **Key Idea:** Use LLMs as **optimizers** for feature-generation rules **with decision tree reasoning**

without language descriptions

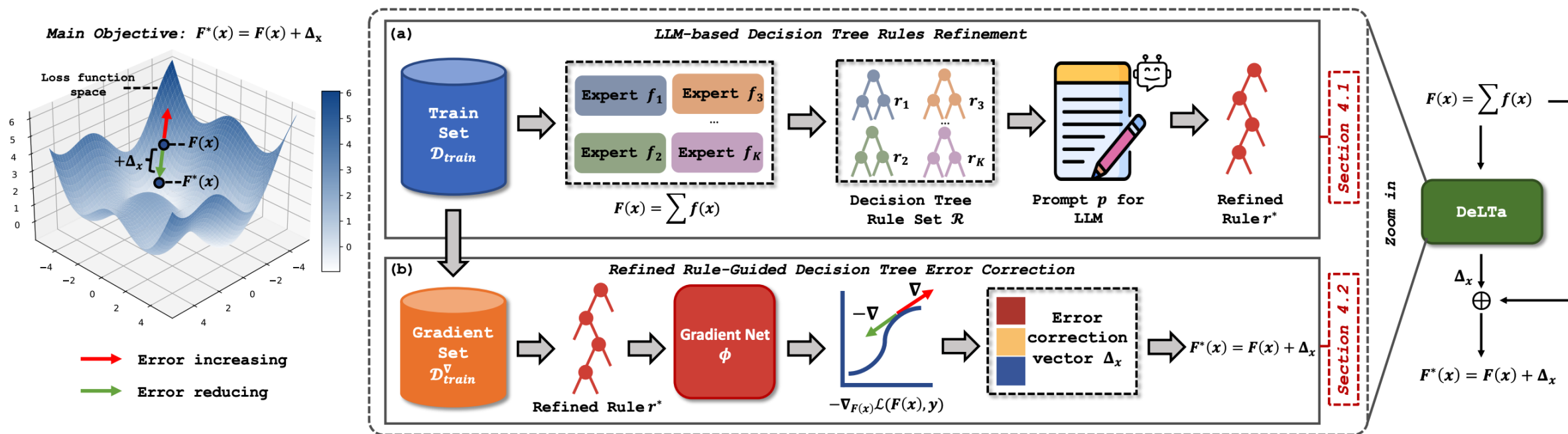
Dataset	XGBoost		MLP		HyperFast	
	Baseline	OCTree (Ours)	Baseline	OCTree (Ours)	Baseline	OCTree (Ours)
electricity	8.32±0.0	6.65 ±0.1 (20.1%)	15.64±0.3	14.82 ±0.4 (5.2%)	15.25±0.5	14.70 ±0.5 (3.6%)
rl	23.61±0.8	19.32 ±0.4 (18.2%)	32.03±4.2	28.30 ±1.7 (11.6%)	33.77±1.3	33.50 ±1.2 (0.8%)
compass	22.91±0.5	18.89 ±0.4 (17.6%)	27.41±1.0	26.78 ±0.1 (2.3%)	25.74±0.6	24.91 ±1.1 (3.2%)
coverttype	9.10±0.2	7.96 ±0.0 (12.5%)	8.73±0.4	8.25 ±0.3 (5.5%)	9.86±1.6	9.21 ±1.3 (6.6%)
phoneme	10.89±0.5	10.15 ±0.7 (6.8%)	12.06±0.8	10.98 ±0.6 (9.8%)	10.55 ±0.7	10.57±0.9 (N/I)
kddCup09	19.86±1.1	19.07 ±1.4 (4.0%)	24.30 ±0.3	24.30 ±1.6 (0.0%)	25.75±0.7	24.46 ±1.1 (5.0%)
pol	1.69±0.2	1.62 ±0.2 (4.0%)	1.37±0.3	1.27 ±0.3 (7.3%)	1.70±0.4	1.55 ±0.2 (8.8%)
Magic	14.25±0.3	13.75 ±0.4 (3.5%)	14.60±0.2	14.50 ±0.0 (0.7%)	14.95±0.2	14.34 ±0.5 (4.1%)
california	9.45±0.6	9.13 ±1.0 (3.4%)	11.91±0.3	11.37 ±0.1 (4.5%)	11.75±0.7	11.02 ±0.6 (6.2%)
house_16H	11.66±0.5	11.32 ±0.2 (3.0%)	13.07±0.2	12.54 ±0.6 (4.1%)	12.77±0.3	12.29 ±0.4 (3.8%)
eye_movements	35.06±0.7	34.17 ±2.0 (2.6%)	40.03±1.2	39.86 ±1.9 (0.4%)	41.33±1.5	40.29 ±1.7 (2.5%)
road-safety	21.14±0.0	20.65 ±0.1 (2.3%)	22.17±0.4	21.87 ±0.1 (1.4%)	24.54±0.3	24.07 ±0.4 (1.9%)
kdd_ipums_la	10.89±1.0	10.69 ±1.0 (1.8%)	13.13±1.3	11.72 ±1.5 (10.7%)	16.15±0.3	13.55 ±1.4 (16.1%)
MiniBooNE	5.48±0.2	5.42 ±0.1 (1.2%)	9.69±0.3	7.35 ±0.2 (24.1%)	6.61±0.4	6.54 ±0.2 (1.1%)
credit	22.02±0.3	21.78 ±0.3 (1.1%)	24.43±0.6	23.23 ±0.7 (4.9%)	25.06±1.1	24.30 ±1.8 (3.0%)
Higgs	27.95±0.7	27.91 ±0.2 (0.1%)	29.43±0.4	28.80 ±0.2 (2.1%)	30.04±0.2	29.73 ±0.5 (1.0%)
jannis	20.61 ±0.1	20.64±0.1 (N/I)	22.28 ±0.1	22.51±0.1 (N/I)	24.29±0.4	23.65 ±0.3 (2.6%)
wine	19.11 ±3.3	19.18±3.9 (N/I)	21.53 ±3.1	21.59±1.4 (N/I)	19.18 ±2.7	19.31±2.2 (N/I)
bank-marketing	20.09 ±0.3	20.31±0.6 (N/I)	21.11±0.4	21.09 ±0.4 (0.1%)	21.25 ±1.0	21.66±0.8 (N/I)

Feature Generation with Decision Tree

- **Key Idea:** Use **LLMs as optimizers** for feature-generation rules **with decision tree reasoning**
 - LLMs with decision tree reasoning successfully generate meaningful features **with and without language descriptions**
- Why decision tree reasoning helps?
 - Decision trees provide compact feedback to the LLM
 - Validation score alone is weak feedback
 - Tree rules provide interpretable reasoning about previous feature attempts
 - LLM can refine future feature rules using this feedback

DeLTa: LLM Meets Decision Trees

- **Key Idea:** Use an LLM to generate a rule for enhancing decision tree
 - Ask the LLM to generate a refined rule from multiple decision-trees
 - Use the rule to calibrate the prediction of original decision trees



DeLTa: LLM Meets Decision Trees

- **Key Idea:** Use an LLM to generate a rule for enhancing decision tree
 - Ask the LLM to generate a refined rule from multiple decision-trees
 - Use the rule to calibrate the prediction of original decision trees
- Relying on only decision tree rules, not semantic priors of column names

# Num	Method (LLM used)	Additional requirements				Datasets						
		Tab	Extra	Feat	Tune	BL ↑	CR ↑	Car ↑	BA ↑	AD ↑	JA ↑	Average ↑
All	TabLLM (T0)	✓	–	✓	✓	0.761	<u>0.737</u>	0.845	0.906	<u>0.864</u>	<u>0.691</u>	<u>0.801</u>
All	LIFT (GPT-3.5)	✓	–	✓	✓	0.689	0.691	0.729	0.825	0.81	0.647	0.732
All	TP-BERTa (RoBERTa)	✓	✓	✓	✓	0.761	0.730	0.826	0.916	0.844	0.659	0.789
All	GTL (LLaMA2)	✓	✓	✓	✓	N/A	N/A	N/A	N/A	N/A	N/A	N/A
All	LIFT-ICL (GPT-4o)	✓	–	✓	–	N/A	N/A	N/A	N/A	N/A	N/A	N/A
All	P2T (GPT-4o)	✓	✓	✓	–	N/A	N/A	N/A	N/A	N/A	N/A	N/A
All	FeatLLM (GPT-4o)	✓	–	✓	–	<u>0.768</u>	0.701	0.589	0.887	0.842	0.540	0.721
All	DeLTa (GPT-4o)	–	–	–	–	0.829	0.783	<u>0.836</u>	<u>0.908</u>	0.868	0.705	0.822
256	TabLLM (T0)	✓	–	✓	✓	0.344	<u>0.690</u>	<u>0.471</u>	<u>0.785</u>	0.787	0.373	0.575
256	LIFT (GPT-3.5)	✓	–	✓	✓	0.651	0.682	0.431	0.724	0.773	0.398	0.610
256	TP-BERTa (RoBERTa)	✓	✓	✓	✓	0.721	0.677	0.408	0.853	0.772	0.464	<u>0.649</u>
256	GTL (LLaMA2)	✓	✓	✓	✓	N/A	N/A	N/A	N/A	N/A	N/A	N/A
256	LIFT-ICL (GPT-4o)	✓	–	✓	–	N/A	N/A	N/A	N/A	N/A	N/A	N/A
256	P2T (GPT-4o)	✓	✓	✓	–	N/A	N/A	N/A	N/A	N/A	N/A	N/A
256	FeatLLM (GPT-4o)	✓	–	✓	–	<u>0.729</u>	0.658	0.459	0.740	0.772	<u>0.473</u>	0.639
256	DeLTa (GPT-4o)	–	–	–	–	0.732	0.713	0.736	0.767	<u>0.778</u>	0.476	0.700

Summary: LLMs as Semantic Components

- **Main strategy:** Use LLMs to improve tabular models rather than directly predict every sample
 - Generate semantic features
 - Optimize feature-generation and decision rules
- **Practical benefits:**
 - Avoids serializing and querying the LLM for every test row
 - Reduces inference cost and latency
 - Retains the efficiency of standard tabular models
- **LLMs can inject semantic knowledge** into efficient tabular pipelines without serving as the final predictor

Main Questions for This Lecture

- Can an LLM directly predict from tabular data?
 - Serialize rows as text and use the LLM as a predictor
- Can an LLM improve another tabular predictor?
 - Use semantic knowledge for feature engineering or hybrid pipelines
- **Can an LLM help detect semantically abnormal samples?**
 - Use likelihood scoring, restored context, or semantic evaluation
- Can we build an LLM-based tabular foundation model?
 - Pre-train on many tables and transfer to unseen tasks

LLMs for Tabular Anomaly Detection

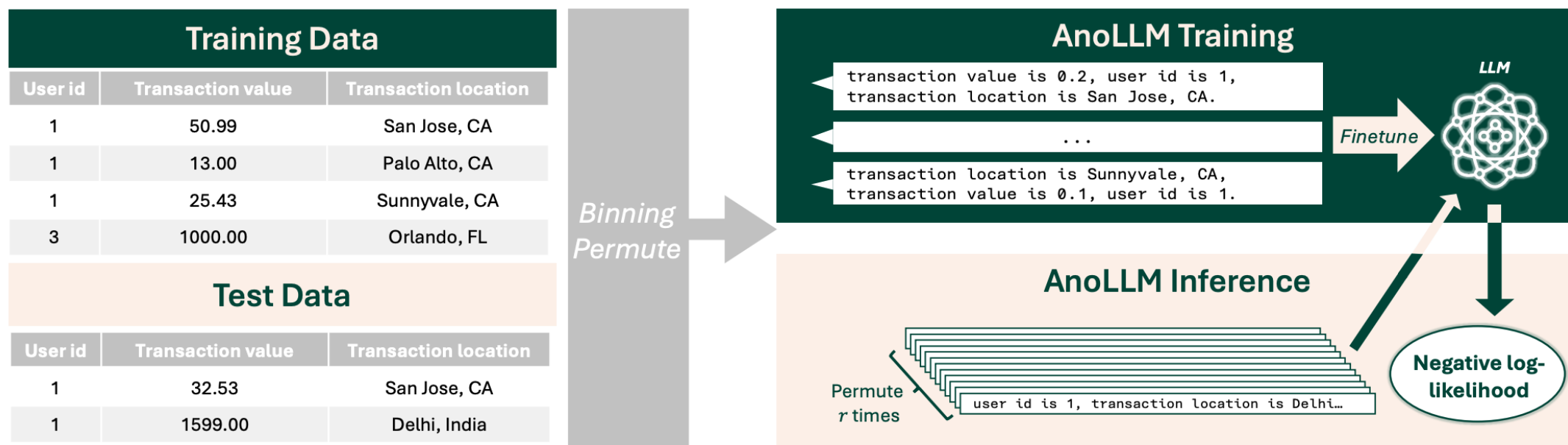
- Anomalies are often **domain-specific** and **context-dependent**
 - Rare values are not always anomalous
 - Common values can be suspicious in the wrong context
- **Traditional AD models mostly rely on raw feature values**
 - They may miss semantic inconsistencies
 - They often ignore column descriptions, units, and domain knowledge

Key Question:

Can LLMs help define what is abnormal using semantic context?

LLM Likelihood for Anomaly Scoring

- **Key Idea:** Use a language model to model normal tabular rows
 - Convert tabular rows into standardized text
 - Fine-tune a pre-trained LLM on normal data
 - Low likelihood $\rightarrow$ abnormal data



LLM Likelihood for Anomaly Scoring

- **Key Idea:** Use a language model to model normal tabular rows
 - Convert tabular rows into standardized text
 - Fine-tune a pre-trained LLM on normal data
 - Low likelihood → abnormal data
- AnoLLM can handle mixed-type tabular data
- **Limitations**
 - Requires fine-tuning LLM on normal data
 - Inference can be expensive
 - Likelihood may not always align with semantic abnormality

ReTabAD: Restoring Semantic Context

- **Key Idea:** Evaluate how metadata helps tabular anomaly detection
 - **Problem:** Existing tabular AD benchmarks mainly provide raw feature values
 - They lack textual metadata such as feature descriptions and domain knowledge

Resource Type	Coverage (number)		Data Type		Algorithm Type			Information Used	
	Datasets	Algorithms	Tabular data	Metadata	ML	DL	LLM	Statistic	Semantic
Benchmark									
DAMI Repository (Campos et al., 2016)	23	12	✓	✗	✓	✗	✗	✓	✗
ADBench (Han et al., 2022)	57	14	✓	✗	✓	✓	✗	✓	✗
Library									
PyOD (Zhao et al., 2019)	-	10+	✓	✗	✓	✓	✗	✓	✗
DeepOD (Xu et al., 2023a)	-	9	✓	✗	✗	✓	✗	✓	✗

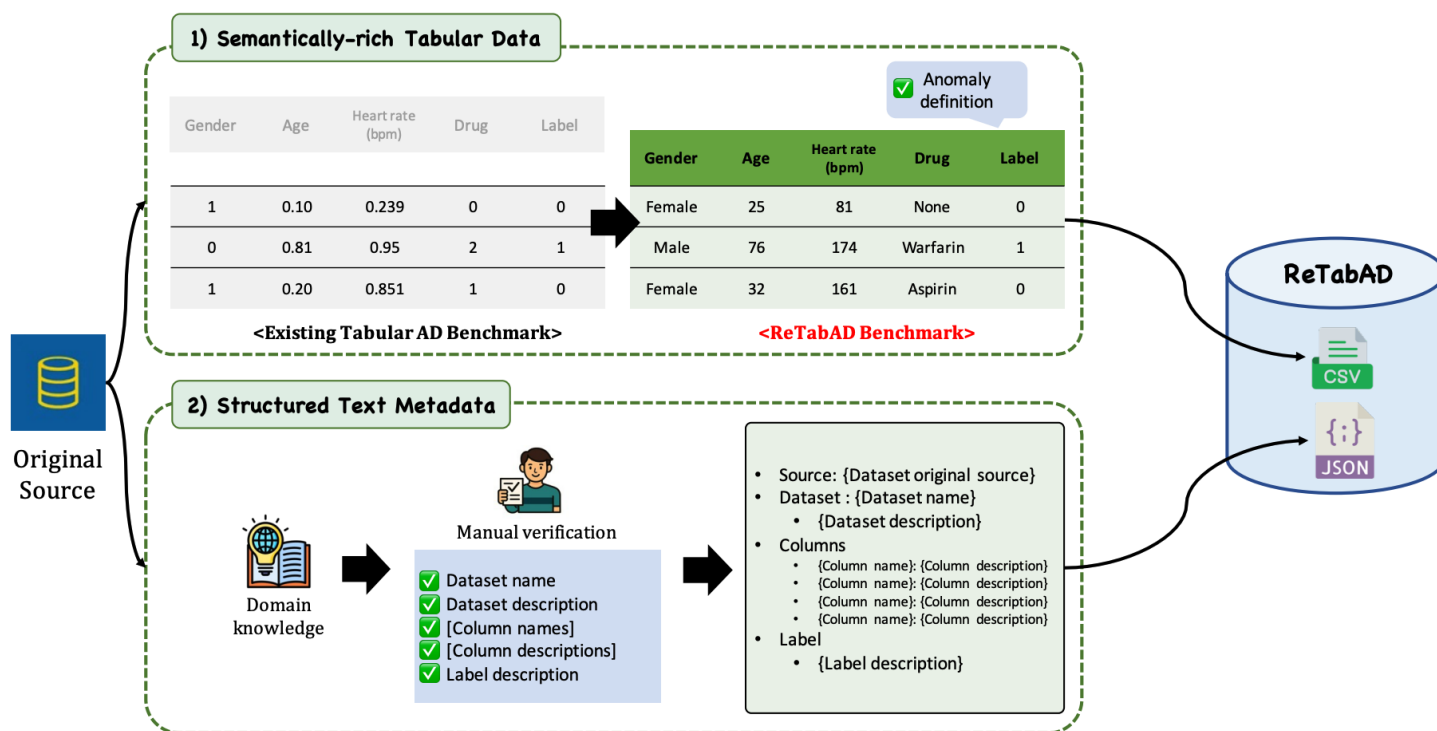
ReTabAD: Restoring Semantic Context

- **Key Idea:** Evaluate **how metadata helps tabular anomaly detection**
 - **Problem:** Existing tabular AD benchmarks mainly provide raw feature values
 - They lack textual metadata such as feature descriptions and domain knowledge
 - This paper builds a benchmark with **semantically enriched tabular datasets**

Resource Type	Coverage (number)		Data Type		Algorithm Type			Information Used	
	Datasets	Algorithms	Tabular data	Metadata	ML	DL	LLM	Statistic	Semantic
Benchmark									
DAMI Repository (Campos et al., 2016)	23	12	✓	✗	✓	✗	✗	✓	✗
ADBench (Han et al., 2022)	57	14	✓	✗	✓	✓	✗	✓	✗
Library									
PyOD (Zhao et al., 2019)	-	10+	✓	✗	✓	✓	✗	✓	✗
DeepOD (Xu et al., 2023a)	-	9	✓	✗	✗	✓	✗	✓	✗
ReTabAD (ours)	20	20	✓	✓	✓	✓	✓	✓	✓

ReTabAD: Restoring Semantic Context

- **Key Idea:** Evaluate how metadata helps tabular anomaly detection
 - **Problem:** Existing tabular AD benchmarks mainly provide raw feature values
 - They lack textual metadata such as feature descriptions and domain knowledge
 - This paper builds a benchmark with **semantically enriched tabular datasets**



Dataset: Cardiotocography

Source: <https://archive.ics.uci.edu/dataset/193/cardiotocography>

Description: Measurements from fetal heart rate (FHR) and uterine contractions (UC) from 2126 cardiotocograms.

Column	Type	Logical Type	Description
LB	int	numerical	FHR baseline (bpm)
AC	float	numerical	Accelerations per second
UC	float	numerical	Uterine contractions per second
ASTV	int	numerical	% time with abnormal short term variability
...	...	...	...
Tendency	string	categorical	Histogram tendency (Left-skewed, Symmetric)

Label: Binary label (0: Normal, 1: Suspect/Pathologic)

ReTabAD: Restoring Semantic Context

- **Key Idea:** Evaluate how metadata helps tabular anomaly detection
 - Semantic metadata enables **context-aware "zero-shot"** anomaly detection

Dataset	Training-based Methods										Zero-shot LLM	
	IForest	OCSVM	LOF	DeepSVDD	NeuTraL	SLAD	DIF	MCM	DRL	AnoLLM	No Desc	Full Desc
automobile	0.648	0.645	0.688	0.845	0.810	0.780	0.647	0.801	0.762	0.625	0.806	0.767
backdoor	0.888	0.876	0.702	0.800	0.948	0.935	0.973	0.961	0.902	0.885	0.804	0.827
campaign	0.732	0.700	0.669	0.721	0.760	0.717	0.677	0.761	0.737	0.738	0.596	0.813
cardiotocography	0.830	0.868	0.826	0.813	0.791	0.764	0.740	0.843	0.847	0.735	0.643	0.829
census	0.620	0.647	0.596	0.712	0.686	0.687	0.726	0.729	0.728	0.737	0.662	0.895
churn	0.506	0.651	0.465	0.573	0.651	0.571	0.494	0.580	0.553	0.557	0.499	0.809
cirrhosis	0.849	0.823	0.843	0.807	0.831	0.810	0.820	0.830	0.814	0.850	0.694	0.850
covertype	0.968	0.998	0.980	0.971	0.998	0.999	0.998	0.998	0.995	0.990	0.861	0.961
credit	0.639	0.717	0.590	0.638	0.690	0.691	0.689	0.671	0.680	0.495	0.501	0.681
equip	0.980	0.987	0.987	0.984	0.985	0.981	0.987	0.986	0.984	0.986	0.975	0.978
gallstone	0.650	0.671	0.704	0.731	0.780	0.758	0.649	0.757	0.745	0.574	0.615	0.667
glass	0.944	0.959	0.974	0.963	0.968	0.952	0.936	0.966	0.949	0.907	0.832	0.919
glioma	0.722	0.725	0.711	0.766	0.784	0.886	0.686	0.790	0.764	0.708	0.586	0.895
quasar	0.915	0.930	0.966	0.804	0.954	0.952	0.793	0.950	0.897	0.957	0.816	0.961
seismic	0.710	0.710	0.615	0.729	0.716	0.727	0.736	0.716	0.714	0.746	0.646	0.741
stroke	0.679	0.716	0.625	0.722	0.703	0.678	0.705	0.753	0.727	0.687	0.562	0.776
vertebral	0.500	0.638	0.540	0.581	0.697	0.679	0.340	0.615	0.477	0.597	0.461	0.755
wbc	0.961	0.969	0.960	0.969	0.785	0.938	0.679	0.957	0.954	0.871	0.861	0.968
wine	0.987	0.957	0.974	0.974	0.988	0.980	0.760	0.979	0.958	0.933	0.663	0.991
yeast	0.838	0.875	0.803	0.820	0.841	0.846	0.815	0.856	0.841	0.809	0.731	0.861
Average Rank	7.50	5.35	7.70	6.40	4.30	5.50	7.85	3.95	6.30	7.35	10.80	4.10
Average AUROC	0.778	0.803	0.761	0.796	0.818	0.817	0.742	0.825	0.801	0.769	0.691	0.847

ReTabAD: Restoring Semantic Context

- **Key Idea:** Evaluate how metadata helps tabular anomaly detection
 - Semantic metadata enables **context-aware "zero-shot"** anomaly detection
 - Also enables more **interpretable reasoning**

Cirrhosis.json

Metadata:

```
"dataset_name": "cirrhosis",
"description": "The dataset includes clinical features for predicting survival
states of patients with liver cirrhosis.",
"columns": [
{
"name": "Prothrombin",
"dtype": "float",
"logical_type": "numerical",
"description": "Prothrombin time in seconds"
},
...
"label_description": "In this dataset, the normal/abnormal classification is
based on patient survival status."
```

Type A (w/ $C_{statistic}$)

GT label: 1 (Anomaly)

Anomaly score: 0.6

Reasoning: **Feature AP** is above the normal range.

Type D (w/ $C_{statistic}$, $C_{feature}$, C_{domain})

GT label: 1 (Anomaly)

Anomaly score: 0.68

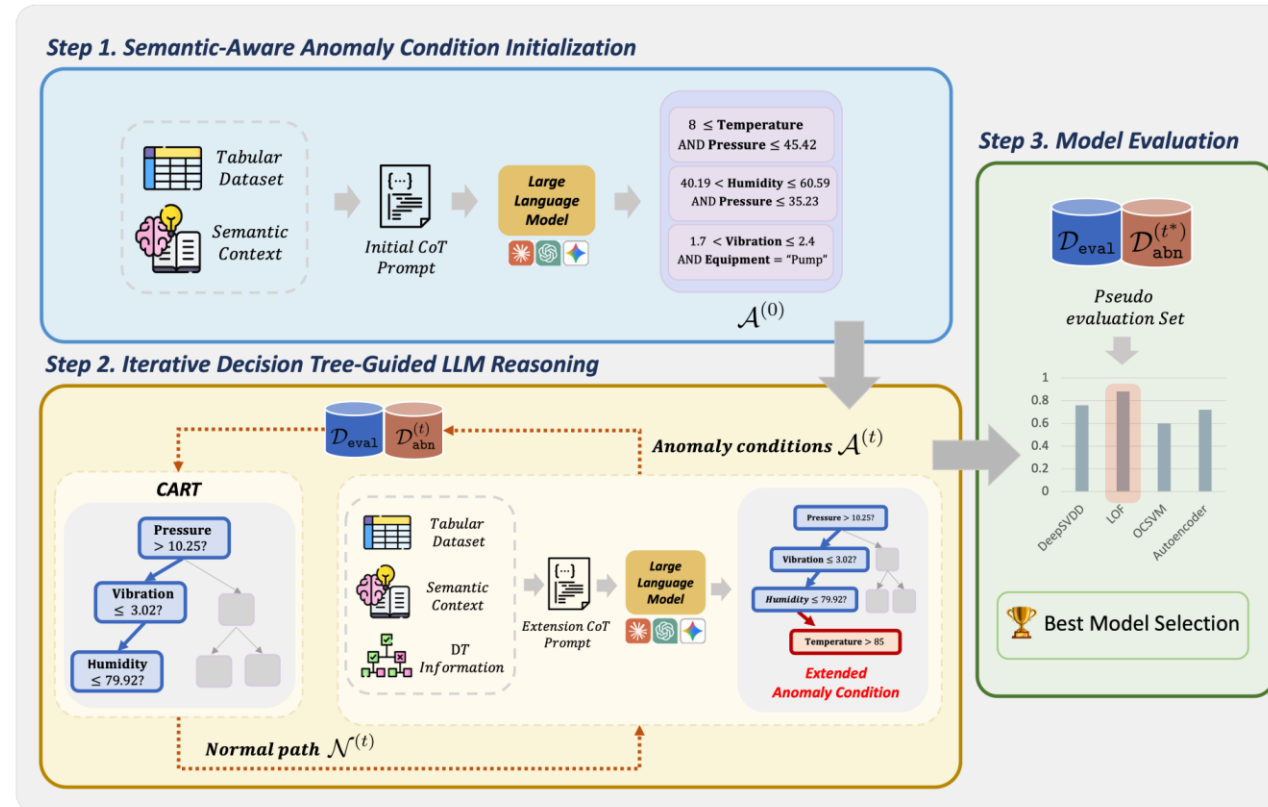
Reasoning: The elevated **Prothrombin time** (11.50) is the key anomaly, indicating **compromised liver synthetic function**. While other markers are currently stable, impaired coagulation is a direct measure of liver failure and a significant predictor of **mortality**, warranting an abnormal classification.

AutoAnoEval: Model Selection w/o Labels

- **AD model selection is difficult** when anomaly labels are unavailable
 - Reliable evaluation requires both normal and anomalous samples
 - True anomalies may be absent even in validation sets
 - Different detectors may perform well on different types of anomalies
- **Question:** Can we generate abnormal samples for model evaluation?
 - Generated anomalies should be **semantically meaningful**
 - They should also be **diverse**, ranging from obvious outliers to subtle anomalies near the normal boundary

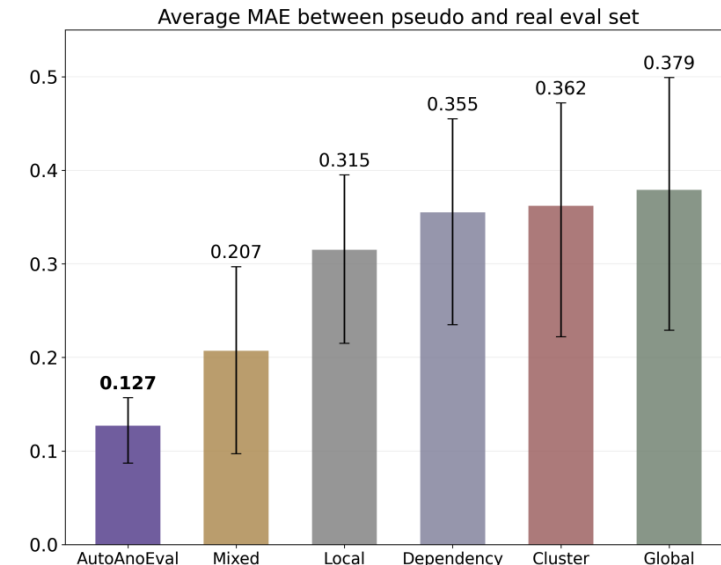
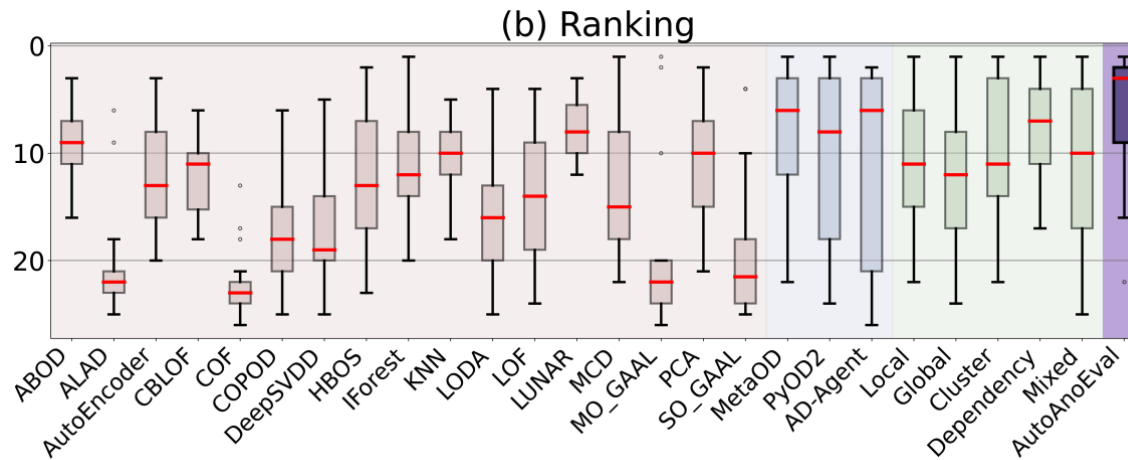
AutoAnoEval: Model Selection w/o Labels

- **Key Idea:** Generate pseudo-anomaly via tree-guided LLM reasoning
 - Normal decision paths can be extracted by decision trees
 - LLM refines anomaly conditions by modifying the normal decision paths



AutoAnoEval: Model Selection w/o Labels

- **Key Idea:** Generate pseudo-anomaly via tree-guided LLM reasoning
 - Normal decision paths can be extracted by decision trees
 - LLM refines anomaly conditions by modifying the normal decision paths
- The generated evaluation set **offers reasonable performance estimates** that closely approximate those on real-world anomalies



Summary: Tabular Anomaly Detection

- LLMs can support tabular AD in different ways:
 - **Anomaly scoring:** Model normal rows as text and detect unlikely samples
 - **Context-awareness:** Use semantic metadata to reason about abnormality
 - **Model selection:** Generate meaningful pseudo-anomalies for evaluation
- Why are LLMs useful?
 - Anomalies are often defined by domain context, not only by statistical rarity
 - LLMs can use feature descriptions and domain knowledge

LLMs extend tabular anomaly detection
from value-based scoring to semantic reasoning and evaluation.

Main Questions for This Lecture

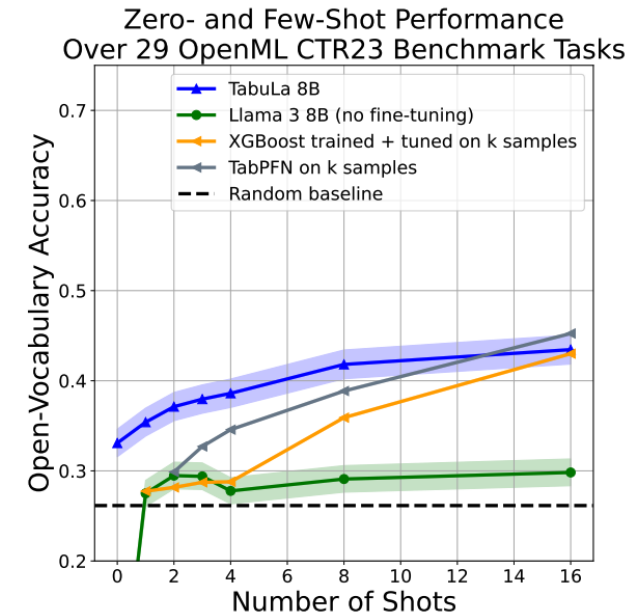
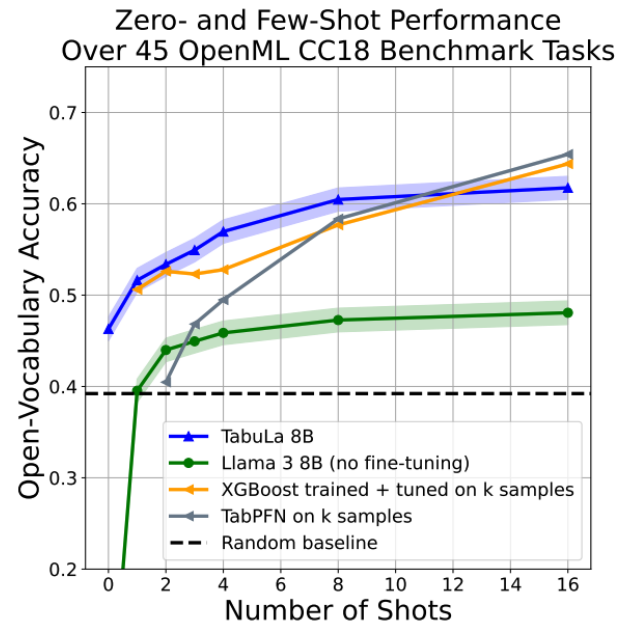
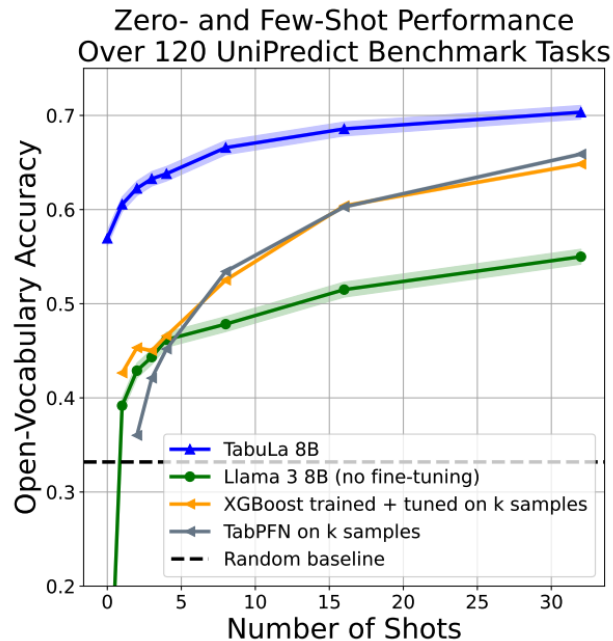
- Can an LLM directly predict from tabular data?
 - Serialize rows as text and use the LLM as a predictor
- Can an LLM improve another tabular predictor?
 - Use semantic knowledge for feature engineering or hybrid pipelines
- Can an LLM help detect semantically abnormal samples?
 - Use likelihood scoring, restored context, or semantic evaluation
- **Can we build an LLM-based tabular foundation model?**
 - Pre-train on many tables and transfer to unseen tasks

Toward Tabular Foundation Models

- Can we build a **tabular foundation model** that can transfer across many tables and tasks?
- Why is this difficult?
 - Tables are highly heterogeneous
 - Unlike text, tabular tasks are not naturally standardized
 - Large-scale training requires unifying many tables into a common format
- **Two directions:**
 - LLM-based tabular foundation models
 - TabPFN-style foundation models (next lecture)

TabuLa-8B: LLM-based Tabular FM

- **Key Idea:** Fine-tune a large language model on a massive corpus of serialized tabular prediction tasks
 - Base model: Llama 3-8B
 - Training dataset (T4): 2.1B rows from over 4M unique tables
 - Evaluation: 329 unseen datasets



Summary

- **LLMs can exploit semantic information in tables**, but direct prediction is not always practical
- **LLMs enable broader applications in tabular ML**
 - Semantic feature engineering and rule refinement
 - Context-aware anomaly detection
 - Foundation models transferable across tables and tasks
- **The key is using semantic knowledge** while preserving the efficiency and reliability of tabular modeling

Thank You for Your Attention!
